LICENCIATURA DE ENGENHARIA INFORMATICA

RELATÓRIO TICKET2HELP

LUIS PINTO

MIGUEL PINHEIRO

RICARDO FREIXO

PROGRAMAÇÃO AVANÇADA

VILA NOVA DE GAIA
JUNHO 2024

SAA-IMP-048.01

Índice

Índice III

Índice de figuras V

1. Introdução 1

1.1. Resumo da Proposta 1

1.2. Requisitos Funcionais 1

1.2.1 Gestão de Acessos 1

1.2.2 Geração de Tickets 1

1.2.3 Atendimento de Tickets 2

1.2.4 Listagem de Tickets 2

1.2.5 Geração de Gráficos 2

1.3. Requisitos não funcionais 2

1.3.1 Desempenho 2

1.3.2 Segurança 2

1.3.3 Manutenibilidade 2

1.3.4 Usabilidade 2

2. Estrutura do Sistema 1

2.1 Componentes de Sistema 1

3. Cenário de utilização 2

4. Extra 6

5. Testes Unitários implementados 7

6. Acessibilidade digital 8

7. Dificuldades encontradas 9

8. Conclusão 9

9. Referências e Bibliografia 10

10. Evidências de trabalho colaborativo no Github 11

Índice de figuras

Figura 1 Diagrama de classes 1
Figura 2- Diagrama de componentes 1
Figura 3 Página Login 2
Figura 4 Página principal do utilizador 3
Figura 5 Pop-up de confirmação de envio de ticket 3
Figura 6 Página principal de admin 4
Figura 7 Página de resposta do ticket 4
Figura 8 Pop-up confirmação de resposta 5
Figura 9 Página de visualização de tickets feito pelo user 5
Figura 10 Pop-up de confirmação de ticket respondido 5
Figura 11 Exemplo de placeholder 6
Figura 12 Código de testes unitários 7
Figura 13 Pop-up com símbolo respetivo 8
Figura 14 Evidencias de trabalho de colaborativo 11

1. Introdução

Este relatório documenta o desenvolvimento de um sistema de gestão de tickets de helpdesk para a empresa Ticket2Help. O sistema foi projetado para atender a solicitações de serviços de hardware (HW) e serviços de software (SW), adotando uma arquitetura em 3 camadas (UI, BLL, DAL) e implementado seguindo o padrão de design MVC (Model-View-Controller) utilizando a plataforma WPF (Windows Presentation Foundation) para a interação com o utilizador. O objetivo principal é oferecer uma solução eficiente e intuitiva para a gestão de tickets, facilitando o atendimento e resolução de problemas de hardware e software dentro da empresa.

1.1. Resumo da Proposta

A proposta consiste em criar uma plataforma de helpdesk interna que permita aos utilizadores criar e acompanhar tickets de solicitação de serviços de hardware e software, enquanto os técnicos de helpdesk poderão atender a esses tickets e gerar relatórios. A aplicação garante a gestão de acessos através de um sistema de login, assegurando que cada utilizador possua as permissões adequadas às suas funções.

1.2. Requisitos Funcionais

1.2.1 Gestão de Acessos

- Implementação de um sistema de login para garantir que apenas utilizadores autorizados possam aceder à plataforma.
- Diferenciação de permissões entre técnicos de helpdesk e outros utilizadores.

1.2.2 Geração de Tickets

- Possibilidade de criar tickets para serviços de hardware ou software.
- Inclusão de informações automáticas como número sequencial, data e hora de geração, código do colaborador e estado inicial do ticket ("porAtender").
- Para tickets de hardware: registro de equipamento e descrição da avaria.
- Para tickets de software: registro do software e descrição da necessidade

1.2.3 Atendimento de Tickets

- Visualização e atualização de tickets "porAtender" pelos técnicos de helpdesk.
- Registro automático da data e hora do atendimento.
- Inclusão de informações específicas do atendimento realizadas pelo técnico.
- Visualização dos tickets gerados pelo utilizador, com detalhes e estados.

1.2.4 Listagem de Tickets

- Visualização dos tickets gerados pelo utilizador, com detalhes e estados.

1.2.5 Geração de Gráficos

- Possibilidade de gerar relatórios detalhados de tickets atendidos.

1.3. Requisitos não funcionais

1.3.1 Desempenho

- O sistema deve ser responsivo e permitir acesso rápido e eficiente aos dados.

1.3.2 Segurança

- Garantia de segurança dos dados através de gestão de acessos e permissões apropriadas.
- Proteção dos dados sensíveis dos utilizadores e técnicos.

1.3.3 Manutenibilidade

- Estrutura de código clara e bem documentada para facilitar futuras manutenções e expansões.
- Implementação de testes unitários para assegurar a qualidade do código e prevenir falhas.

1.3.4 Usabilidade

- Interface intuitiva e amigável, facilitando a interação dos utilizadores com o sistema.
- Navegação simples e clara entre as diferentes funcionalidades\

2. Estrutura do Sistema

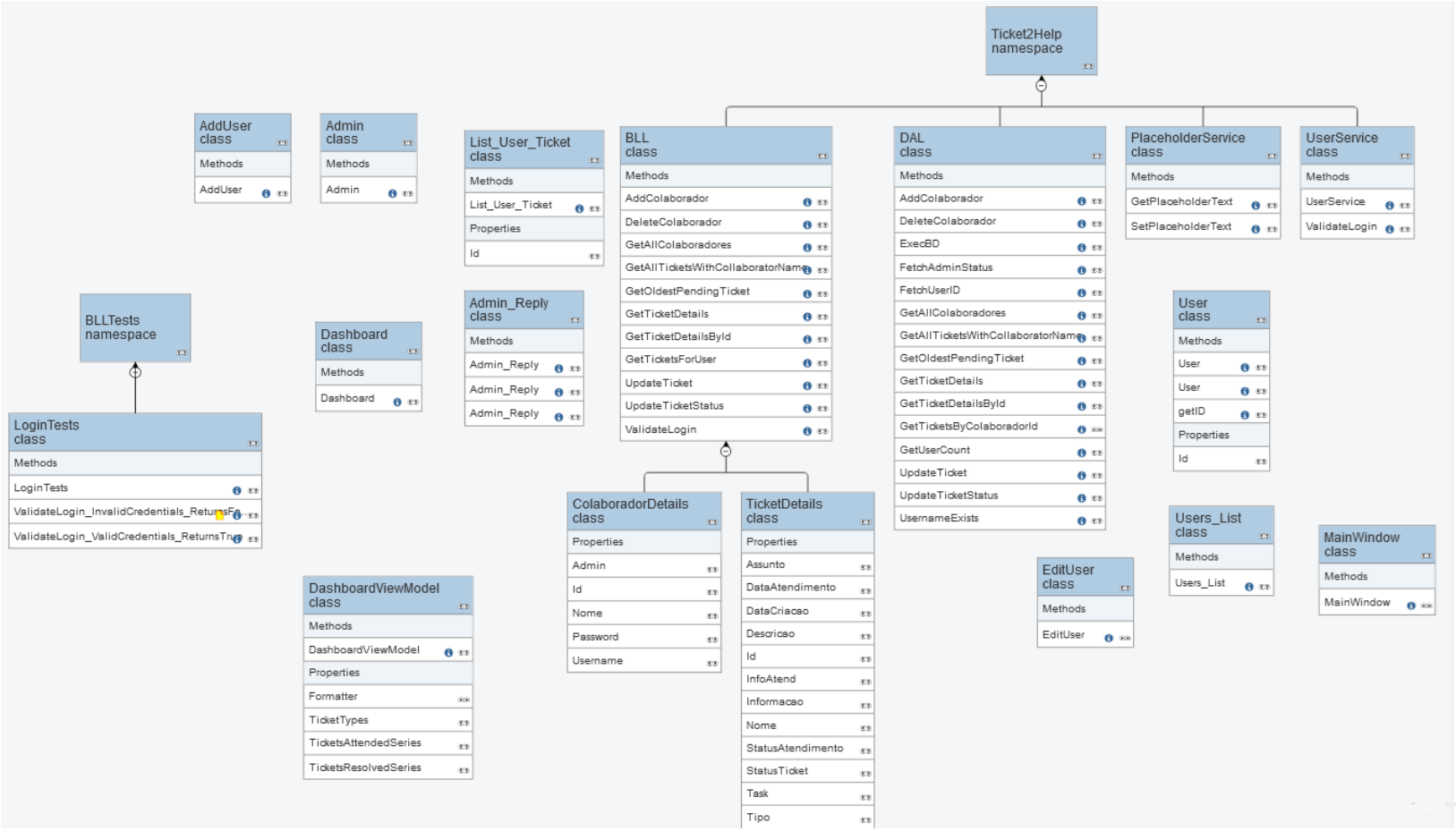

Figura 1 Diagrama de classes

2.1 Componentes de Sistema

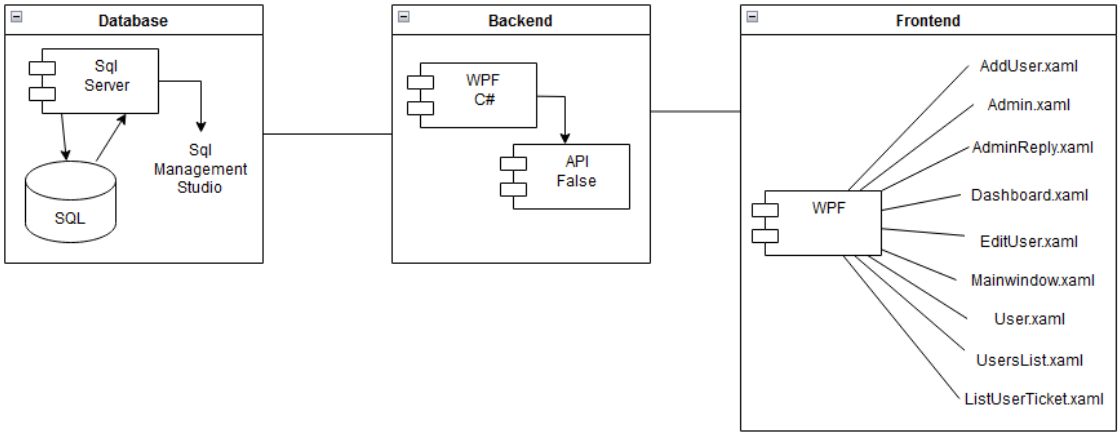

Figura 2- Diagrama de componentes

3. Cenário de utilização

Um colaborador da empresa necessita de um monitor novo, para isso ele vai abrir um ticket usando a aplicação Ticket2Help.

Para começar este processo este colaborador necessitará de dar login na plataforma.

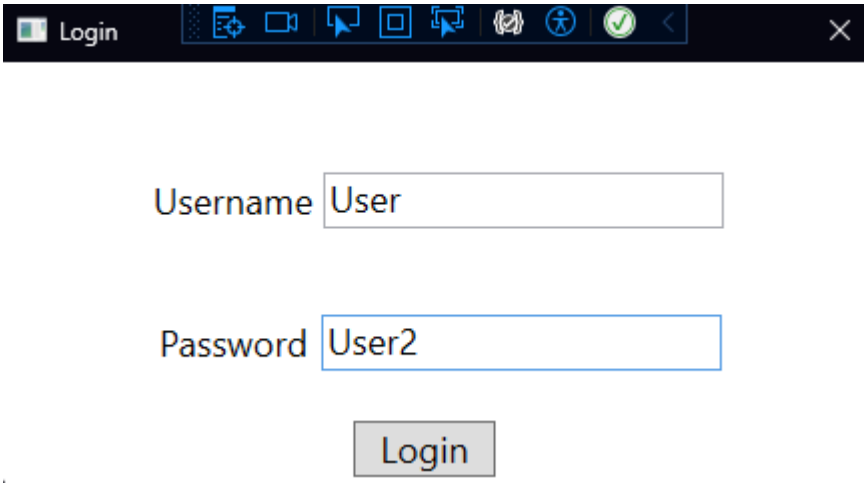

Figura 3 Página Login

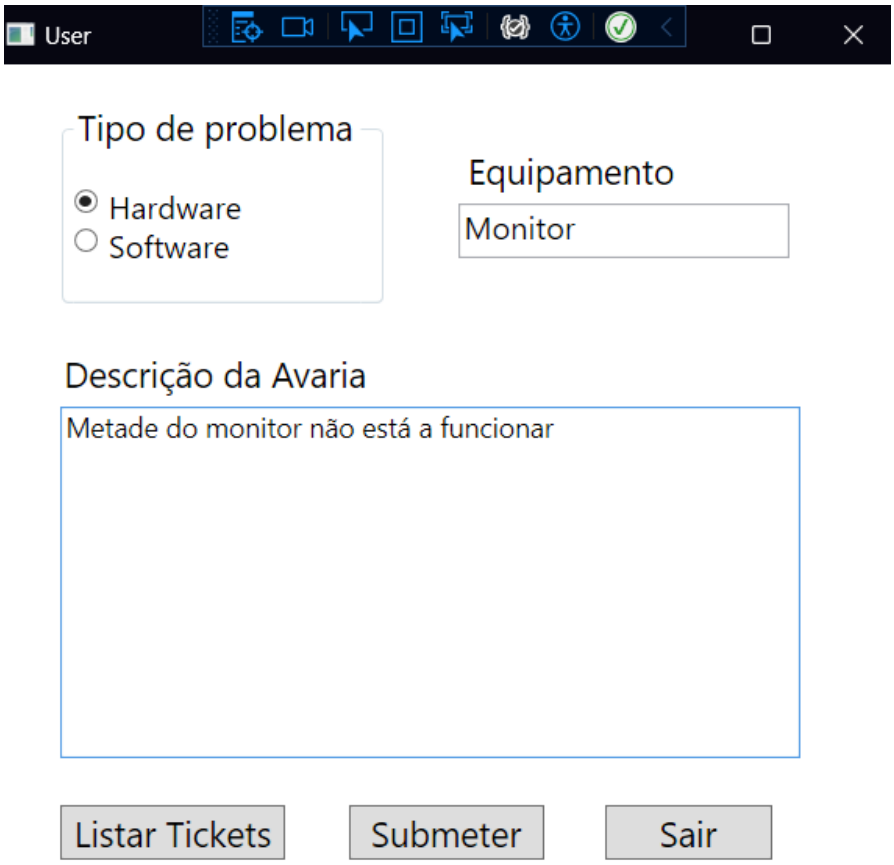

Figura 4 Página principal do utilizador

Após o login ele vai descrever o problema usando o pequeno formulário em WPF para que o admin resolva o problema deste utilizador e ao submeter este terá uma confirmação de que o ticket foi, de facto, enviado.

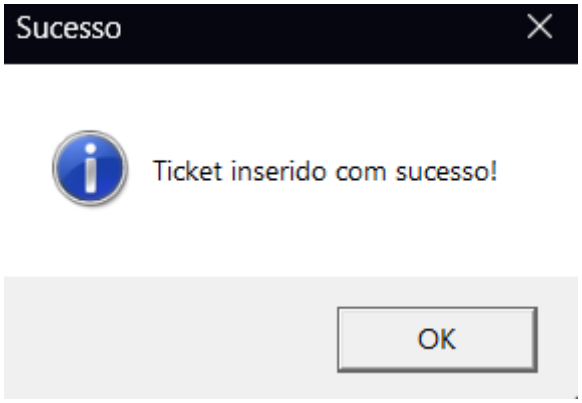

Figura 5 Pop-up de confirmação de envio de ticket

Figura 6 Página principal de admin

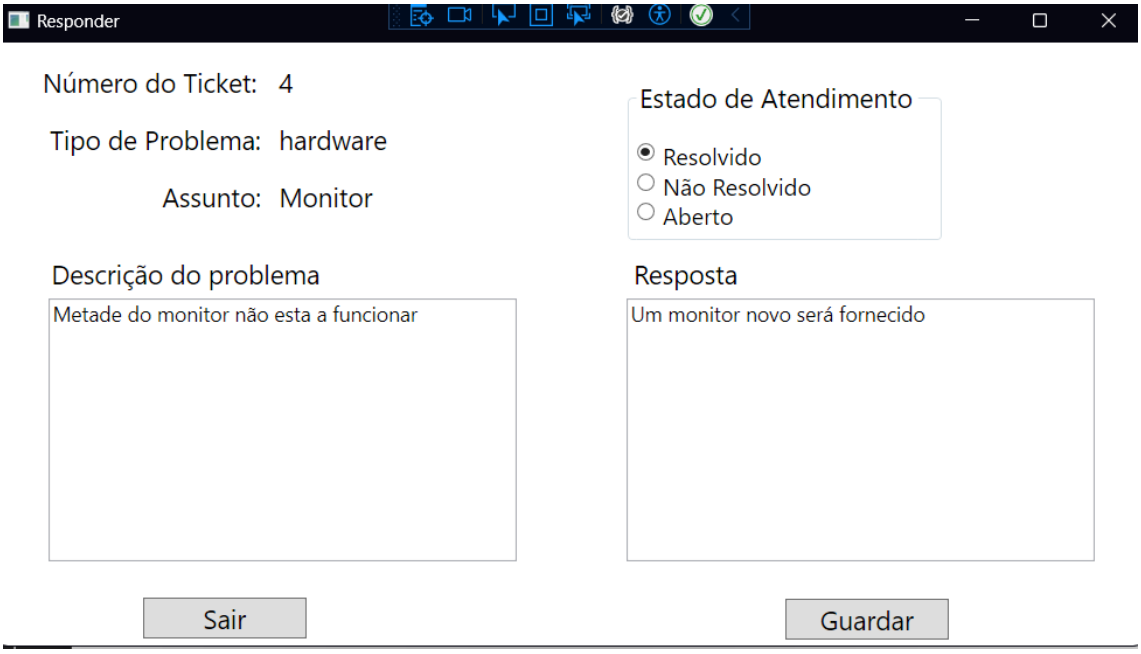

Figura 7 Página de resposta do ticket

Após o colaborador enviar o ticket este vai ser recebido pelo admin que irá responder ao ticket.

Este ao clicar guardar ira receber o pop-up a confirmar que foi respondido com sucesso

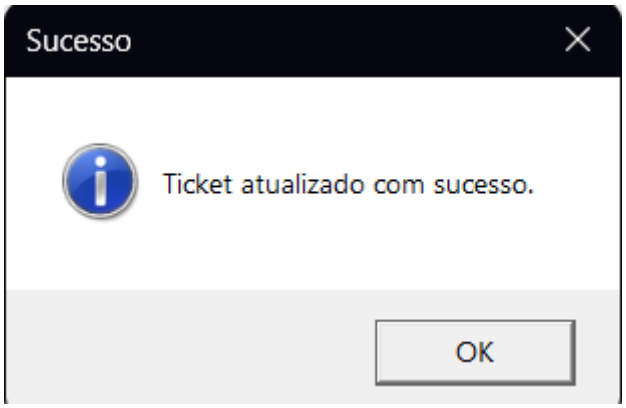

Figura 8 Pop-up confirmação de resposta

Mais tarde o colaborador pode verificar o estado do seu ticket, assim tendo a informação se já foi atendido ou não.

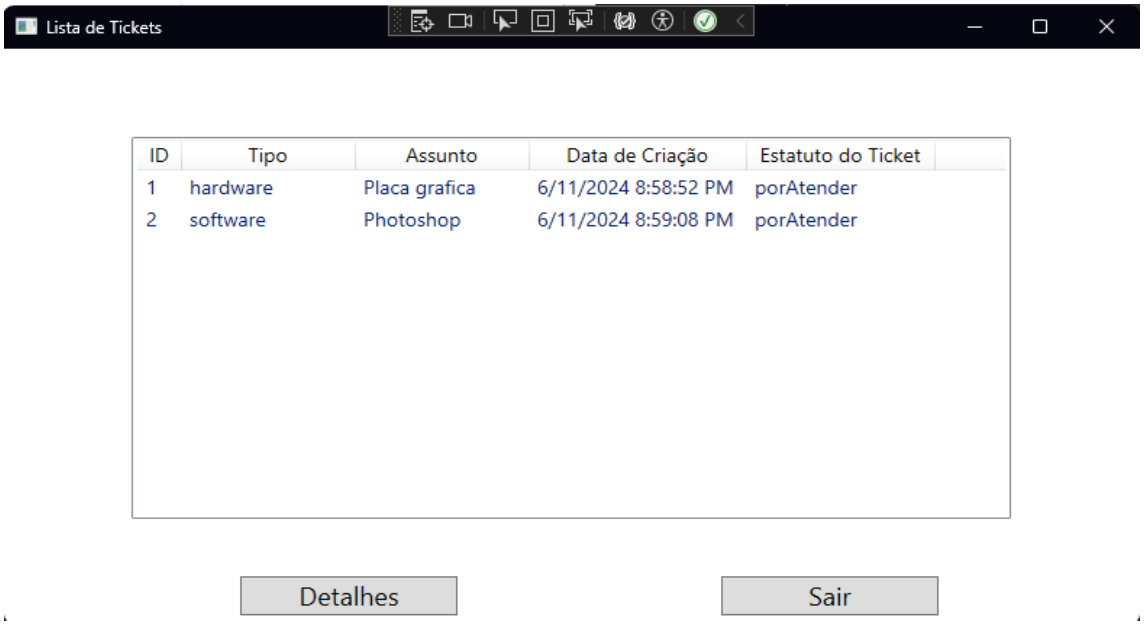

Figura 9 Página de visualização de tickets feito pelo user

4. Extra

Como extras para este projeto foi decido inserir placeholders em todas as caixas de texto para facilitar a usabilidade da aplicação. Para isto foi desenvolvido umas funções chamada textbox_lostfocus e textbox_gotfocus que insere ou remove o placeholder , estando a parte gráfica no xaml como estilo.

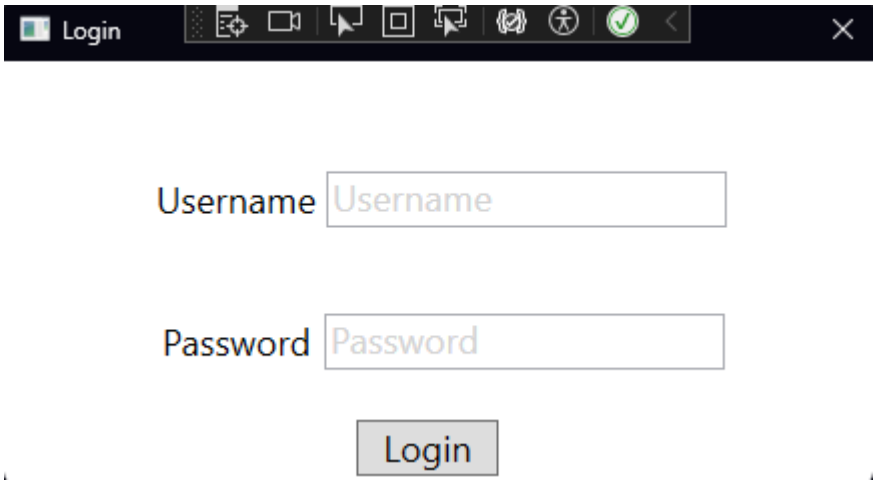

Figura 11 Exemplo de placeholder

5. Testes Unitários implementados

Os testes de login descritos foram realizados utilizando a biblioteca Moq. Dois cenários principais foram avaliados para o serviço de login da aplicação Ticket2Help: primeiro, validando se as credenciais corretas retornam um valor verdadeiro, e segundo, verificando se as credenciais incorretas retornam um valor falso. No primeiro teste, um moq do Data Access Layer (DAL) foi configurado para retornar um utilizador válido, e o resultado foi positivo, confirmando a validade das credenciais. No segundo teste, o moq do DAL foi configurado para não encontrar nenhum utilizador com as credenciais fornecidas, resultando corretamente em uma falha de autenticação. Em ambos os casos, as interações com o moq foram verificadas para garantir que os métodos esperados foram chamados exatamente uma vez.

```
public LoginTests()
{
    _mockDal = new Mock<IUserDAL>();
    _userService = new UserService(_mockDal.Object);
}

/// <summary>
/// Testa o método ValidateLogin com credenciais válidas.
/// </summary>
[Fact]
0 references
public void ValidateLogin_ValidCredentials_ReturnsTrue()
{
    string username = "validUser";
    string password = "validPassword";

    _mockDal.Setup(dal => dal.GetUserCount(username, password)).Returns(1);

    bool result = _userService.ValidateLogin(username, password);

    Assert.True(result);
    _mockDal.Verify(dal => dal.GetUserCount(username, password), Times.Once);
}

/// <summary>
/// Testa o método ValidateLogin com credenciais inválidas.
/// </summary>
[Fact]
0 references
public void ValidateLogin_InvalidCredentials_ReturnsFalse()
{
    string username = "invalidUser";
    string password = "invalidPassword";

    _mockDal.Setup(dal => dal.GetUserCount(username, password)).Returns(0);

    bool result = _userService.ValidateLogin(username, password);

    Assert.False(result);
    _mockDal.Verify(dal => dal.GetUserCount(username, password), Times.Once);
}
```

Figura 12 Código de testes unitários

6. Acessibilidade digital

Para facilitar a utilização do software todo o texto tem pelo menos 14 pixéis de tamanho, toda a informação de sucesso ou erro e acompanhada com o ícone respetivo para fácil interpretação pelo utilizador.

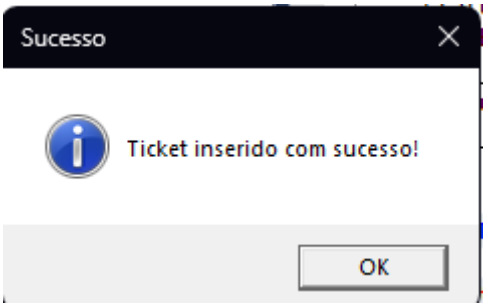

Figura 13 Pop-up com símbolo respetivo

7. Dificuldades encontradas

Implementar um dashboard eficaz no contexto do sistema, apresentou diversas dificuldades. A seguir estão algumas dessas dificuldades:

Extração de Dados Relevantes

A camada BLL contém várias classes (Ticket, TicketHardware, TicketSoftware, etc.) que manipulam diferentes tipos de tickets. Extrair dados relevantes de cada uma dessas classes e combiná-los em um formato coeso e útil para o dashboard requer um entendimento profundo da estrutura dos dados e das inter-relações entre as classes.

Comunicação entre Camadas

A implementação do dashboard exige uma integração robusta entre a camada de apresentação (Ticket2Help), a camada de lógica de negócios (BLL) e a camada de acesso a dados (DAL). Garantir que os dados sejam corretamente extraídos do DAL, processados pelo BLL e apresentados de forma coerente no dashboard pode ser complexo e propenso a erros.

8. Conclusão

O desenvolvimento do sistema de gestão de tickets de helpdesk para a Ticket2Help resultou em uma solução eficiente e organizada para o atendimento de solicitações de serviços de hardware e software. Com uma arquitetura em 3 camadas e implementado utilizando o padrão MVC e WPF, o sistema garante a segurança e a eficácia na gestão de tickets. Técnicos de helpdesk podem atender rapidamente aos tickets, registrar ações e gerar relatórios detalhados, enquanto os demais utilizadores podem criar e acompanhar suas solicitações. A interface intuitiva e a documentação detalhada, aliadas aos testes unitários, asseguram a qualidade e a manutenibilidade do sistema.

O projeto trouxe benefícios significativos para a Ticket2Help, centralizando as solicitações e reduzindo o tempo de resposta dos atendimentos. A aplicação de padrões de design de software e o uso de ferramentas de colaboração como o GitHub contribuíram para uma estrutura de código clara e uma gestão eficiente das versões. A solução desenvolvida não só melhora a eficiência operacional, mas também aumenta a satisfação dos colaboradores, demonstrando o sucesso das decisões estratégicas e técnicas adotadas ao longo do desenvolvimento.

9. Referências e Bibliografia

https://stackoverflow.com/

https://chatgpt.com/

https://v0.lvcharts.com/App/examples/Wpf/start

10. Evidências de trabalho colaborativo no Github

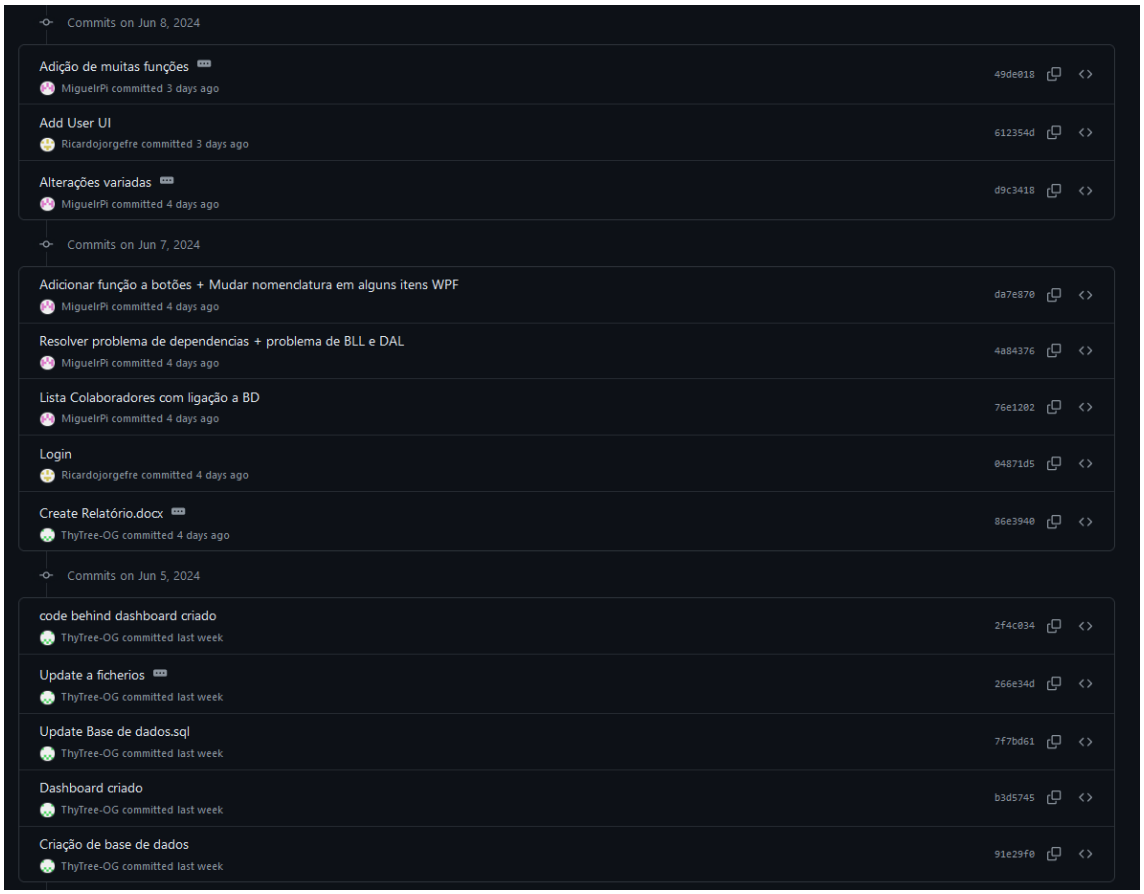

Figura 14 Evidencias de trabalho de colaborativo

https://github.com/ThyTree-OG/Ticket2help/commits/main/